

Hardware-Constrained Architecture Search for Lane-Change Intention Prediction from Vehicle Time Series

Sepehr Mohammady, [TODO: co-authors, order], [TODO: supervisor]

Abstract—Predicting a driver’s lane-change intention early enough to act on it is a time-series classification problem with hard deployment limits: the model has to run on automotive-grade microcontrollers within tight memory and latency budgets. We apply constrained neural architecture search to the Lane Change Intention Recognition driving dataset [3] (windows of 50 samples over 31 signals) and obtain compact one-dimensional convolutional networks for three tasks: three-class intention classification and time-to-lane-change regression for left and right maneuvers. [TODO: results: accuracy, RMSE, model size, latency on STM32] The searched models improve on the published baseline (Transformer, 0.51 s time-to-lane-change RMSE, deployed in FP32 [1]) while using [TODO: X] $\times$ less memory, and we report measured, not estimated, footprint and latency on the same STM32 targets after int8 quantization.

Index Terms—Neural architecture search, lane-change intention, time-series classification, TinyML, embedded inference, STM32.

I. INTRODUCTION

[TODO: Motivation: intention prediction as advance warning for ADAS; why on-device inference (latency, privacy, cost, no connectivity assumptions); the gap: published DMIR models target accuracy, not deployment budgets.]

Contributions:

- 1) a constrained search space and objective tuned to one-dimensional driving signals under microcontroller budgets;
- 2) models that beat the published DMIR baseline on all three tasks at a fraction of its footprint [TODO: verify once results exist];
- 3) measured deployment numbers (flash, RAM, latency) on [TODO: STM32 part], with the full pipeline released as open source.

II. RELATED WORK

A. Lane-change intention prediction

[TODO: The LC dataset and framework baseline [1] (TTL regression, Transformer RMSE 0.51 s, FP32 deployment on STM32H7B3/F401); earlier HighD-based TTL work (RMSE 0.63–0.7); MicroNAS for time series and TinyTNAS as closest NAS relatives.]

The authors are with the ELIOS Lab, Department of Electrical, Electronic and Telecommunications Engineering and Naval Architecture (DITEN), University of Genoa, Genoa, Italy.

This work was supported by the SYNERGIES project [TODO: grant number and official acknowledgment text].

B. Constrained architecture search for microcontrollers

[TODO: muNAS [2], MCUNet/TinyNAS, MicroNets; what transfers to 1D time series and what does not.]

III. DATA

We use the Lane Change Intention Recognition dataset [3] (50 human drivers in the CARLA simulator, 10 Hz, over 3,400 annotated lane changes; MIT license), prepared as fixed windows of $T=50$ samples (5 s) over $F=31$ channels [TODO: confirm channel order and whether channel 31 is fileTime — drop it if so; official feature count is 30]. Classification uses balanced splits of 94,620 / 16,332 / 19,722 windows (train/val/test) over three classes: no intention, right lane change, left lane change. The regression tasks map a window to the time until the lane change, $t \in [0, 4]$ s discretized at 0.1 s, with 39,313 / 7,742 / 8,097 (LCL) and 33,201 / 5,828 / 7,073 (LCR) windows.

The splits are user-independent: we verified by matching raw per-driver session logs against the prepared windows (correlation over 50-sample windows) that the validation and test drivers follow the protocol of [1] with no window leakage across splits. The test splits contain isolated spikes (up to 5×10^6) on the right/left curvatureDx channel pair — a numerical-derivative artefact absent from training; we clip validation and test features to the per-channel training range and leave training data untouched.

IV. METHOD

A. Search space

[TODO: 1D cells: depthwise-separable and standard convolutions, kernel sizes, widths, depth, pooling; operator set restricted to what the ST Edge AI compiler maps efficiently.]

B. Constraints and search algorithm

[TODO: peak RAM, flash after int8 quantization, MACs; aging evolution with constraint handling; search cost budget.]

V. EXPERIMENTS

A. Setup

[TODO: training schedule, hardware, seeds/repeats, hyperparameters; all runs logged to the released experiments.jsonl.]

TABLE I

TEST-SET COMPARISON. THE PUBLISHED BASELINE [1] REPORTS A SINGLE TIME-TO-LANE-CHANGE REGRESSION (TRANSFORMER, ~ 54 K PARAMETERS, FP32); THE INTERNAL-REFERENCE ROW IS UNPUBLISHED RESULTS ON THE SAME PREPARED DATA. MODEL SIZE IS THE INT8 WEIGHT FOOTPRINT.

Model	Size (KB)	Acc. (%)	MAE LCR (s)	MAE LCL (s)
Published Transformer [1]	~ 216	—	0.298 (single TTLC)	
Internal reference (unpub.)	—	92.0	RMSE 0.42/0.44	
Our DSCNN baseline	~ 10	91.5	0.318	0.333
Ours (searched, best)	216	91.6	0.290	0.320
Ours (searched, compact)	21	91.5	0.294	0.321
Ours, no turn-signal (ablation)	5.2	90.1	—	—

B. Results

Table I reports test-set metrics. All searched-model numbers are our own test-set evaluation of the saved models. *These are preliminary*: each search stopped before its target budget and will be re-run to completion. [TODO: replace with final full-search numbers; add peak-RAM, MACs, and measured on-device latency columns from ST Edge AI.]

The searched models improve on the published SOTA MAE (0.290 vs 0.298 for LCR) at a wide range of footprints, down to a few tens of kilobytes. Removing the turn-signal channels (the leaky feature) drops three-class accuracy by under one point (91.6% to 90.8%), and a 5.2 KB model without them still reaches 90.1%—evidence that the model anticipates from vehicle dynamics and surrounding traffic rather than reading a declared signal. [TODO: RMSE columns and the internal-reference RMSE (0.42/0.44) are not yet matched because the search optimizes MAE; decide RMSE-aware objective vs MAE-primary framing.]

C. Deployment on STM32

Target platform: STM32H7B3I-DK discovery kit (Cortex-M7 at 280 MHz, 1.4 MB SRAM, 2 MB flash) — the same MCU family used by the baseline framework [1], which deployed FP32 models only. [TODO: toolchain version, int8 quantization accuracy drop, flash / RAM / latency measured on device, comparison with the baseline’s Table III.]

VI. CONCLUSION

[TODO: two or three sentences; no new claims.]

REFERENCES

- [1] L. Forneris *et al.*, “A deployment-oriented simulation framework for deep learning-based lane change prediction,” *IEEE Signal Process. Lett.*, vol. 33, pp. 136–140, 2026, doi: 10.1109/LSP.2025.3638676.
- [2] E. Liberis, Ł. Dudziak, and N. D. Lane, “ μ NAS: Constrained neural architecture search for microcontrollers,” in *Proc. 1st Workshop Mach. Learn. Syst. (EuroMLSys)*, 2021, pp. 70–79.
- [3] L. Forneris *et al.*, “Lane change intention recognition dataset,” Zenodo, 2025, doi: 10.5281/zenodo.16686054.
- [4] L. Forneris *et al.*, “Setting up a lightweight simulation environment for automated driving dataset collection,” in *Applications in Electronics Pervading Industry, Environment and Society (ApplePies)*, Springer LNEE, 2024, pp. 156–163, doi: 10.1007/978-3-031-84100-2_19.